

INFORME TÉCNICO DE PLAN DE TRABAJO PARA LA CONSTRUCCIÓN DEL SOFTWARE

1. Portada
2. Introducción
3. Descripción del proyecto Ukiyo CO
4. Objetivos
5. Objetivo general
6. Objetivos específicos
7. Descripción y alcance del software
8. Análisis del diseño del software
9. Plan de trabajo para la construcción del software
10. Herramientas de desarrollo
11. Herramientas de versionamiento
12. Versionamiento local y remoto
13. Estrategia de control de versiones
14. Estándares de codificación
15. Estructura del proyecto
16. Metodología de desarrollo
17. Plan de pruebas y control de calidad
18. Cronograma de actividades
19. Conclusiones
20. Referencias bibliográficas

INFORME TÉCNICO DE PLAN DE TRABAJO PARA LA CONSTRUCCIÓN DEL SOFTWARE

Proyecto: Desarrollo de plataforma web para Ukiyo CO

1. Introducción

El presente informe técnico establece el plan de trabajo para la construcción de una plataforma web orientada a la marca de ropa Ukiyo CO. El proyecto surge de la necesidad de disponer de una herramienta tecnológica que permita fortalecer los procesos de comercialización, presentación de productos, gestión de inventario, administración de pedidos y comunicación con los clientes.

La plataforma será desarrollada considerando una arquitectura organizada en componentes de interfaz, lógica de programación y gestión de datos. Para la construcción de la solución se contempla el uso de tecnologías web como HTML5, CSS3 y JavaScript, complementadas con herramientas de desarrollo y control de versiones como Visual Studio Code, Git y GitHub. HTML permite establecer la estructura y el significado del contenido, CSS permite definir su presentación y JavaScript incorpora comportamientos dinámicos e interacción con el usuario (MDN Web Docs, 2026).

El plan de trabajo busca establecer de manera organizada las actividades necesarias para pasar desde el diseño y análisis inicial hasta la construcción, prueba y documentación del software. Asimismo, se establecen criterios para el manejo del código fuente, el control de versiones y la aplicación de estándares de codificación.

La utilización de herramientas de control de versiones resulta fundamental para mantener un historial de los cambios realizados durante el desarrollo. Git es un sistema distribuido de control de versiones que permite administrar diferentes estados del código y recuperar versiones anteriores cuando sea necesario (Git, s. f.). De manera complementaria, GitHub permitirá almacenar el repositorio remoto y facilitar la gestión de ramas, cambios y versiones del proyecto.

2. Descripción del proyecto Ukiyo CO

Ukiyo CO es una propuesta de marca de ropa orientada al segmento de moda urbana y streetwear. Como parte del desarrollo del proyecto formativo, se plantea la construcción de una plataforma web que permita fortalecer la presencia digital de la marca y facilitar la interacción entre los productos ofrecidos y sus potenciales clientes.

La solución tecnológica tendrá como propósito central presentar el catálogo de productos de manera organizada y permitir la gestión de diferentes procesos relacionados con la comercialización. La plataforma estará diseñada para proporcionar una experiencia de navegación clara, adaptable a diferentes dispositivos y coherente con la identidad visual definida para Ukiyo CO.

El sistema contemplará dos perspectivas principales: la del cliente y la del administrador. Desde la perspectiva del cliente se busca facilitar la consulta de productos y la generación de pedidos. Desde la perspectiva administrativa se pretende proporcionar mecanismos para gestionar productos, inventario, usuarios y pedidos.

La propuesta visual de la plataforma conservará los elementos de identidad establecidos para Ukiyo CO, utilizando principalmente una combinación de tonos negros y crema, acompañados por un color de acento púrpura. De esta manera, el desarrollo tecnológico estará relacionado directamente con la propuesta gráfica y comercial de la marca.

3. Objetivos

3.1 Objetivo general

Establecer un plan técnico de trabajo para la construcción de una plataforma web para Ukiyo CO, definiendo las actividades de desarrollo, las herramientas tecnológicas, el sistema de versionamiento y los estándares de codificación necesarios para obtener una solución organizada, funcional y mantenible.

3.2 Objetivos específicos

1. Analizar los requerimientos y componentes definidos para la plataforma web de Ukiyo CO con el propósito de establecer las actividades necesarias para su construcción.
 2. Seleccionar las herramientas de desarrollo y tecnologías apropiadas para la implementación de la interfaz, las funcionalidades y los componentes del sistema.
 3. Implementar un sistema de control de versiones utilizando Git y GitHub para realizar seguimiento a las modificaciones efectuadas durante el desarrollo.
 4. Establecer estándares de codificación que permitan mantener una estructura organizada, legible y consistente en los diferentes componentes del software.
 5. Definir actividades de prueba y validación que permitan verificar el funcionamiento de las principales funcionalidades de la plataforma.
-

4. Descripción y alcance del software

La plataforma web de Ukiyo CO tendrá como alcance principal la digitalización y organización de los procesos relacionados con la presentación y comercialización de los productos de la marca.

El sistema estará compuesto por diferentes módulos y funcionalidades.

4.1 Página de inicio

La página principal permitirá presentar la identidad de Ukiyo CO, mostrar información relevante de la marca y dirigir al usuario hacia las principales secciones de la plataforma.

4.2 Catálogo de productos

El catálogo permitirá visualizar las prendas disponibles, incluyendo información como nombre, imagen, descripción, precio, categoría y disponibilidad.

4.3 Detalle del producto

Cada producto contará con una sección específica donde el usuario podrá consultar información más detallada y seleccionar las características disponibles antes de incorporarlo al carrito.

4.4 Carrito de compras

El carrito permitirá almacenar temporalmente los productos seleccionados por el usuario, consultar cantidades y visualizar el valor acumulado del pedido.

4.5 Registro e inicio de sesión

La plataforma contemplará mecanismos de registro y autenticación para permitir la identificación de los usuarios y establecer diferentes niveles de acceso.

4.6 Gestión de pedidos

El sistema permitirá registrar la información asociada con los pedidos realizados por los clientes y mantener un estado para facilitar su seguimiento.

4.7 Gestión de inventario

El módulo administrativo permitirá controlar la información relacionada con las existencias de productos, facilitando la actualización de cantidades y disponibilidad.

4.8 Panel administrativo

El administrador podrá acceder a funcionalidades destinadas a la gestión de productos, usuarios, inventario y pedidos.

4.9 Comunicación con la marca

La plataforma podrá incorporar mecanismos de contacto directo con Ukiyo CO, incluyendo la integración con WhatsApp como canal complementario de comunicación.

El alcance inicial no contempla necesariamente la implementación completa de todos los servicios de producción asociados a pagos, alojamiento y distribución comercial. Estos componentes podrán ser incorporados posteriormente como parte de una etapa de ampliación del proyecto.

5. Análisis del diseño del software

El diseño desarrollado para Ukiyo CO establece las características visuales y funcionales que servirán como referencia durante la etapa de construcción. El proceso de desarrollo partirá de los prototipos y modelos previamente establecidos, buscando mantener coherencia entre la propuesta gráfica y la implementación tecnológica.

La interfaz se organizará mediante una estructura de navegación que facilite el desplazamiento entre las diferentes secciones. Entre las principales vistas se encuentran la página de inicio, catálogo, detalle del producto, carrito de compras, inicio de sesión y panel administrativo.

La arquitectura inicial se plantea mediante una separación de responsabilidades entre la presentación, la lógica de interacción y la gestión de información. Esta separación permitirá facilitar el mantenimiento del código y la incorporación de nuevas funcionalidades.

En el componente de presentación se utilizarán HTML5 y CSS3 para construir la estructura y el diseño de las interfaces. JavaScript será utilizado para implementar comportamientos dinámicos, validaciones e interacción con los elementos de la página. Para componentes del lado del servidor se contempla el uso de Python, de acuerdo con la evolución técnica del proyecto.

Las tecnologías HTML, CSS y JavaScript cumplen funciones complementarias dentro del desarrollo web: HTML proporciona estructura y semántica, CSS controla presentación y distribución, y JavaScript permite implementar comportamientos dinámicos (MDN Web Docs, 2026).

6. Plan de trabajo para la construcción del software

El plan de trabajo se divide en diferentes etapas con el propósito de organizar progresivamente las actividades de construcción.

Fase	Actividad	Descripción	Resultado esperado
1	Análisis	Revisión de requerimientos y funcionalidades	Requerimientos definidos
2	Diseño	Elaboración y revisión de interfaces	Prototipos aprobados
3	Estructuración	Organización de carpetas y archivos	Proyecto inicial configurado
4	Versionamiento	Configuración de Git y GitHub	Repositorio configurado
5	Front-End	Desarrollo de interfaces	Vistas funcionales
6	Funcionalidades	Implementación de interacciones	Funciones operativas
7	Back-End	Desarrollo de lógica del servidor	Servicios funcionales
8	Base de datos	Organización y conexión de datos	Información estructurada
9	Pruebas	Validación funcional y visual	Errores identificados
10	Correcciones	Ajustes y optimización	Versión corregida
11	Documentación	Elaboración de documentación técnica	Proyecto documentado
12	Versión final	Integración de componentes	Versión estable

El desarrollo se realizará de manera incremental, de modo que cada componente pueda ser revisado antes de incorporarse a la versión estable del proyecto.

7. Herramientas de desarrollo

Para la construcción de la plataforma se seleccionarán herramientas de acuerdo con las necesidades funcionales y técnicas del proyecto.

Herramienta	Función dentro del proyecto
Visual Studio Code	Edición y organización del código fuente
HTML5	Construcción de la estructura de las páginas
CSS3	Diseño, distribución y estilos visuales
JavaScript	Implementación de interacción y funcionalidades dinámicas
Python	Desarrollo de componentes del lado del servidor
Git	Control de versiones local
GitHub	Repositorio remoto y administración del código
Figma	Diseño y prototipado de interfaces

Herramienta	Función dentro del proyecto
Draw.io	Elaboración de diagramas y modelos
Navegador web	Pruebas de funcionamiento y visualización

Visual Studio Code será utilizado como entorno principal de desarrollo debido a que permite centralizar la edición y organización de los diferentes archivos que componen la aplicación.

HTML5, CSS3 y JavaScript constituyen la base tecnológica del componente Front-End. Estas tecnologías permiten estructurar, diseñar e incorporar comportamiento dinámico a las interfaces web (MDN Web Docs, 2026).

Python será contemplado para el desarrollo de componentes Back-End debido a que permite implementar lógica del lado del servidor y establecer mecanismos de comunicación con otros componentes de la aplicación.

8. Herramientas de versionamiento

El proyecto utilizará Git como herramienta principal para el control de versiones local y GitHub como plataforma de almacenamiento y gestión del repositorio remoto.

Git es un sistema distribuido de control de versiones que permite registrar modificaciones realizadas sobre los archivos de un proyecto y mantener diferentes estados del código (Git, s. f.). Esta característica resulta especialmente útil para el proyecto Ukiyo CO, ya que durante el proceso de construcción será necesario realizar modificaciones frecuentes sobre las interfaces y funcionalidades.

GitHub será utilizado como repositorio remoto para almacenar el código fuente y mantener una copia del historial de modificaciones. De acuerdo con la documentación oficial, un repositorio contiene el código, los archivos y el historial de revisiones del proyecto. Además, permite utilizar ramas para trabajar sobre diferentes versiones del código sin afectar directamente la rama principal (GitHub, 2026).

La estrategia de versionamiento permitirá mantener un historial organizado de los cambios y facilitar la recuperación de versiones anteriores cuando se presente algún error durante el desarrollo.

9. Diferencia entre versionamiento local y remoto

El versionamiento local corresponde al control de las versiones realizado directamente desde el equipo de desarrollo. En el proyecto Ukiyo CO será administrado mediante Git. Esto permite registrar cambios, crear ramas y recuperar estados anteriores del proyecto sin depender permanentemente de una conexión a Internet.

El versionamiento remoto corresponde al almacenamiento del repositorio en un servicio externo. En este proyecto se utilizará GitHub. La documentación oficial de GitHub define un repositorio remoto como aquel que se encuentra almacenado en GitHub y no directamente en el computador del desarrollador (GitHub, 2026).

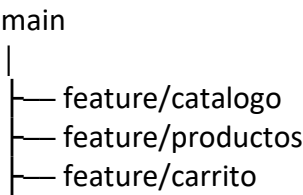
Característica	Versionamiento local	Versionamiento remoto
Herramienta	Git	GitHub
Ubicación	Equipo del desarrollador	Servidor remoto
Registro de cambios	Sí	Sí
Creación de ramas	Sí	Sí
Copia remota	No necesariamente	Sí
Colaboración	Limitada	Facilita colaboración
Recuperación ante pérdida del equipo	Limitada	Mayor disponibilidad
Sincronización	Local	Mediante operaciones de envío y recepción

Por lo tanto, ambas modalidades serán utilizadas de manera complementaria. Git permitirá controlar el desarrollo desde el equipo local y GitHub permitirá mantener una copia remota y administrar el proyecto desde una plataforma centralizada.

10. Estrategia de control de versiones

Para Ukiyo CO se propone utilizar una estrategia basada en ramas. La rama main contendrá las versiones consideradas estables del proyecto. Las nuevas funcionalidades se desarrollarán en ramas independientes antes de ser integradas.

La estructura inicial será:



```
graph TD; A[ ] --- B[feature/login]; A --- C[feature/inventario]; A --- D[feature/admin];
```

Cada rama estará asociada con una funcionalidad específica. Por ejemplo, la rama `feature/catalogo` se utilizará para desarrollar y modificar el catálogo de productos.

Una vez terminada una funcionalidad, se realizarán pruebas y posteriormente se integrarán los cambios a la rama principal. Este procedimiento permite mantener aislados los cambios durante el desarrollo.

GitHub establece que las ramas permiten trabajar sobre diferentes versiones del código y desarrollar funcionalidades sin modificar directamente la rama principal hasta que los cambios estén listos para integrarse (GitHub, 2026).

11. Proceso para registrar cambios

El flujo básico de trabajo será:

```
graph TD; A[Modificar código] --> B[Revisar cambios]; B --> C[git add]; C --> D[git commit]; D --> E[git push]; E --> F[Repositorio GitHub]; F --> G[Revisión]; G --> H[Integración a main];
```

Los mensajes de commit deberán describir claramente el cambio realizado.

Ejemplos:

feat: agregar catálogo de productos
feat: implementar carrito de compras
fix: corregir validación del formulario

style: ajustar diseño responsive
docs: actualizar documentación

El uso de mensajes descriptivos facilitará la consulta posterior del historial del proyecto.

12. Estándares de codificación

Para garantizar la organización y mantenibilidad del código se establecerán estándares de codificación.

12.1 Nombres de archivos

Los nombres deberán ser descriptivos y mantener una estructura coherente.

Ejemplos:

index.html
catalogo.html
producto.html
carrito.html
login.html
admin.html

12.2 Nombres de variables

Las variables deberán utilizar nombres relacionados con la información que almacenan.

Ejemplo:

nombreProducto
precioProducto
cantidadProducto
usuarioActual

12.3 Organización del código

Los archivos HTML, CSS y JavaScript deberán mantenerse separados para facilitar su administración.

No se deberá incluir código CSS extenso directamente dentro de los documentos HTML cuando pueda mantenerse en hojas de estilo externas.

12.4 Indentación

El código deberá utilizar una indentación uniforme que facilite su lectura y mantenimiento.

12.5 Comentarios

Los comentarios deberán utilizarse para explicar segmentos de código cuya finalidad no resulte evidente, evitando comentarios innecesarios.

12.6 Reutilización

Se evitará duplicar código cuando una función o componente pueda reutilizarse en diferentes partes del sistema.

12.7 Validación

Los formularios y datos ingresados por los usuarios deberán validarse antes de ser procesados.

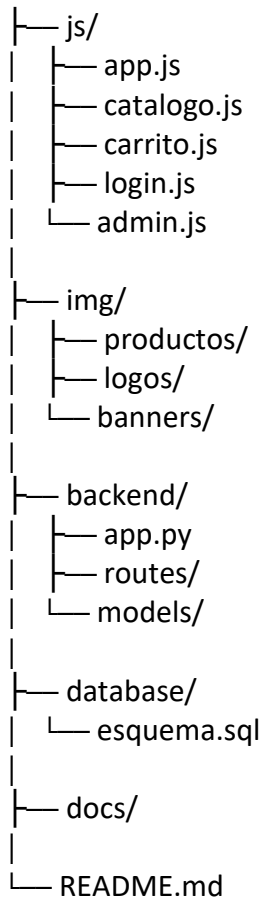
12.8 Organización de estilos

Los estilos se organizarán mediante clases y estructuras coherentes para facilitar modificaciones posteriores.

13. Estructura del proyecto

La estructura inicial propuesta para el proyecto es la siguiente:

```
UkiyoCO/
|
|— index.html
|— catalogo.html
|— producto.html
|— carrito.html
|— login.html
|— registro.html
|— admin.html
|
|— css/
|   |— estilos.css
|   |— responsive.css
|   |— componentes.css
|
```



Esta organización permitirá separar los diferentes componentes del sistema y facilitar el mantenimiento del código.

14. Metodología de desarrollo

Para el proyecto se propone utilizar un enfoque incremental basado en entregas progresivas. Esta estrategia permitirá construir la plataforma por módulos y validar cada componente antes de continuar con el siguiente.

Inicialmente se desarrollará la estructura principal de la plataforma. Posteriormente se incorporarán el catálogo, los productos, el carrito, los usuarios y los componentes administrativos.

El desarrollo incremental permitirá detectar errores de manera temprana y realizar ajustes antes de que los componentes sean integrados completamente.

La metodología propuesta contempla ciclos de trabajo compuestos por:

1. Planificación.
2. Desarrollo.
3. Pruebas.
4. Corrección.
5. Versionamiento.
6. Integración.
7. Documentación.

Cada ciclo generará una versión actualizada del proyecto que será registrada mediante Git.

15. Plan de pruebas y control de calidad

Las pruebas tendrán como objetivo comprobar que las funcionalidades implementadas respondan correctamente a los requerimientos definidos.

Módulo	Prueba	Resultado esperado
Inicio	Navegación	Las secciones cargan correctamente
Catálogo	Visualización	Los productos aparecen correctamente
Producto	Consulta	La información se muestra correctamente
Carrito	Agregar producto	El producto se incorpora al carrito
Carrito	Cantidad	La cantidad puede modificarse
Carrito	Total	El valor se actualiza correctamente
Registro	Validación	El sistema valida los datos
Login	Autenticación	El usuario puede acceder
Inventario	Actualización	Las existencias se modifican
Administración	Gestión	El administrador puede administrar información
Responsive	Diferentes dispositivos	La interfaz se adapta correctamente

También se realizarán pruebas visuales en diferentes tamaños de pantalla para verificar la adaptación de la interfaz.

Las herramientas de desarrollo de los navegadores podrán utilizarse para identificar errores en HTML, CSS y JavaScript, así como para analizar el comportamiento de la aplicación. MDN señala que los navegadores modernos incluyen herramientas de desarrollo útiles para depuración y pruebas durante la construcción de sitios web (MDN Web Docs, 2026).

16. Cronograma de actividades

Etapas	Actividad	Tiempo estimado
1	Análisis de requerimientos	2 días
2	Revisión del prototipo	2 días
3	Configuración del entorno	1 día
4	Configuración de Git y GitHub	1 día
5	Desarrollo de estructura HTML	3 días
6	Implementación de estilos CSS	3 días
7	Desarrollo de funcionalidades JavaScript	4 días
8	Desarrollo de componentes Back-End	4 días
9	Integración de base de datos	3 días
10	Desarrollo del panel administrativo	3 días
11	Pruebas funcionales	3 días
12	Corrección de errores	2 días
13	Documentación	2 días
14	Revisión de versión final	1 día

El cronograma podrá ajustarse de acuerdo con el avance real del proyecto, la complejidad de las funcionalidades y los resultados obtenidos durante las pruebas.

17. Consideraciones de seguridad y mantenimiento

Durante la construcción de la plataforma se tendrán en cuenta principios básicos de seguridad y mantenimiento del software.

Los datos de los usuarios deberán manejarse de manera responsable y no se deberán almacenar credenciales directamente dentro del código fuente. Las variables sensibles, claves y configuraciones privadas deberán mantenerse fuera de los archivos públicos del proyecto.

Asimismo, se deberá evitar almacenar información innecesaria dentro del repositorio y se recomienda utilizar archivos de configuración adecuados para los diferentes entornos.

La estructura modular permitirá realizar modificaciones sin afectar completamente el sistema. Esta característica será importante durante las etapas posteriores del proyecto, cuando puedan incorporarse nuevas funcionalidades como métodos de pago, notificaciones, reportes o integración con servicios externos.

18. Resultados esperados

Como resultado del plan de trabajo se espera obtener una plataforma web funcional para Ukiyo CO que permita presentar los productos de la marca y apoyar los procesos básicos de comercialización.

La solución deberá contar con una interfaz organizada y coherente con la identidad visual de la marca, funcionalidades de navegación, catálogo, productos, carrito y gestión de usuarios, además de componentes administrativos relacionados con productos, inventario y pedidos.

Desde el punto de vista técnico, se espera mantener un código organizado mediante estándares de codificación y un historial de modificaciones administrado mediante Git y GitHub.

El resultado final deberá corresponder con los requerimientos y diseños establecidos durante las etapas anteriores del proyecto.

19. Conclusiones

El establecimiento de un plan de trabajo permite organizar de manera estructurada las actividades necesarias para la construcción de la plataforma web de Ukiyo CO. La definición de fases, herramientas, estándares y mecanismos de control facilita el seguimiento del proceso de desarrollo y permite reducir los riesgos asociados con modificaciones no controladas del código.

La selección de HTML5, CSS3 y JavaScript para el desarrollo del Front-End proporciona una base adecuada para construir la estructura, presentación e interacción de la plataforma. De igual manera, la incorporación de Python para componentes del lado del servidor permitirá ampliar las funcionalidades de la solución de acuerdo con las necesidades del proyecto.

La utilización conjunta de Git y GitHub permitirá diferenciar y complementar el control de versiones local y remoto. Git permitirá registrar los cambios realizados en el equipo de desarrollo, mientras que GitHub permitirá mantener el repositorio remoto y administrar ramas y versiones del proyecto. Esta estrategia contribuye a mantener un historial organizado y facilita la recuperación de versiones anteriores.

Finalmente, la definición de estándares de codificación, estructura de archivos, metodología de desarrollo y plan de pruebas permitirá que el proyecto avance de manera

organizada y facilite futuras actividades de mantenimiento, ampliación y actualización de la plataforma.

20. Referencias

Git. (s. f.). *Git documentation*. <https://git-scm.com/docs/git>

GitHub. (2026). *About repositories*. GitHub Docs. <https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories>

GitHub. (2026). *Hello World*. GitHub Docs. <https://docs.github.com/en/get-started/start-your-journey/hello-world>

MDN Web Docs. (2026). *Aprende desarrollo web*. Mozilla Developer Network. https://developer.mozilla.org/es/docs/Learn_web_development

MDN Web Docs. (2026). *La web y los estándares web*. Mozilla Developer Network. https://developer.mozilla.org/es/docs/Learn_web_development/Getting_started/Web_standards/The_web_standards_model